

Thank you. We have gathered all the necessary requirements to design your enterprise-grade, scalable, and data-driven automation framework.

Here is the architectural blueprint, project layout, and full implementation.

1. Requirement Summary

Ecosystem: Java with Maven as the build and dependency management tool.

Data-Driven Engine: Apache POI reading from external Excel (.xlsx) files.

Design Pattern: Standard Page Object Model (POM) for clear separation of UI elements and test logic.

Execution & Reporting: TestNG for parallel execution, coupled with Extent Reports for interactive HTML reporting and automatic failure screenshots.

Driver & Thread Management: Thread-safe ThreadLocal<WebDriver> execution engine using explicit waits for reliable element synchronization.

2. Architecture and Design Decisions

The framework is architected using a layered approach to isolate responsibilities:

Core Architectural Pillars

Thread Safety (ThreadLocal): To run tests in parallel without cross-thread interference, the WebDriver instance is wrapped inside a ThreadLocal object. This ensures each thread isolates its browser session.

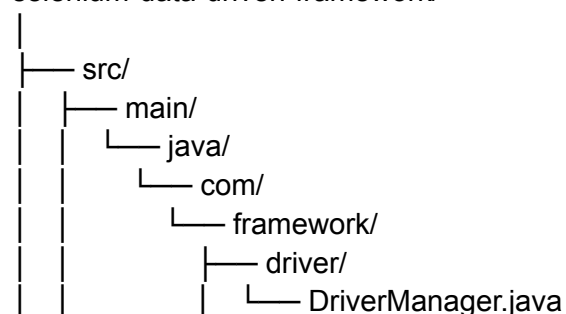
Data Isolation: Test logic contains zero hardcoded values. The @DataProvider bridge fetches rows from Excel sheets via Apache POI and feeds them dynamically into TestNG test methods.

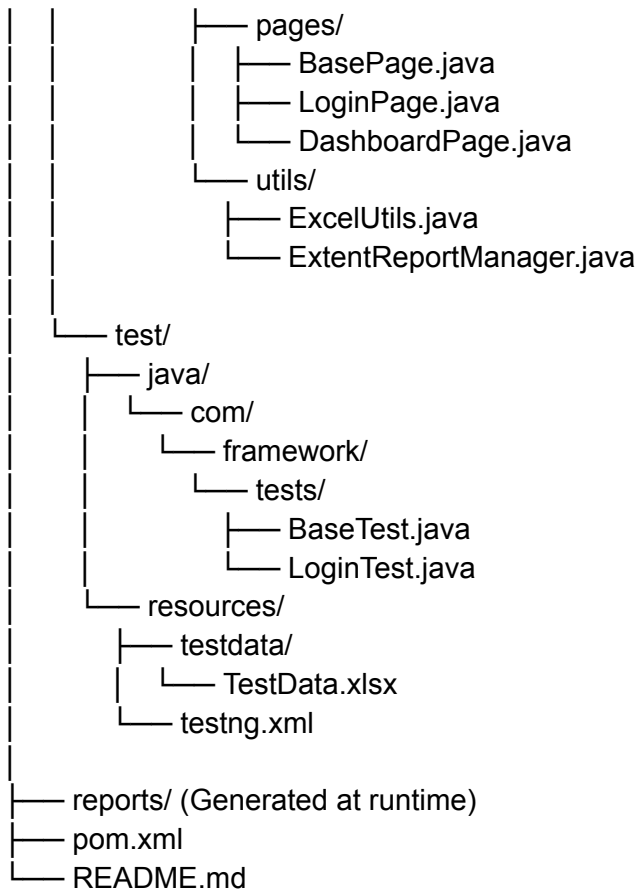
Explicit Synchronization: Implicit waits are avoided entirely. We utilize WebDriverWait matching specific ExpectedConditions to completely eliminate timing-based flakiness.

3. Recommended Project Structure

Plaintext

selenium-data-driven-framework/





4. Complete Framework Implementation

Step 1: Configuration (pom.xml)

This file manages all necessary dependencies and configures parallel suite execution via the maven-surefire-plugin.

XML

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.framework</groupId>
  <artifactId>selenium-data-driven-framework</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>
```

```

<dependencies>
  <dependency>
    <groupId>org.seleniumhq.selenium</groupId>
    <artifactId>selenium-java</artifactId>
    <version>4.18.1</version>
  </dependency>

  <dependency>
    <groupId>org.testng</groupId>
    <artifactId>testng</artifactId>
    <version>7.9.0</version>
    <scope>test</scope>
  </dependency>

  <dependency>
    <groupId>org.apache.poi</groupId>
    <artifactId>poi-ooxml</artifactId>
    <version>5.2.5</version>
  </dependency>

  <dependency>
    <groupId>com.aventstack</groupId>
    <artifactId>extentreports</artifactId>
    <version>5.1.1</version>
  </dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>3.2.5</version>
      <configuration>
        <suiteXmlFiles>
          <suiteXmlFile>src/test/resources/testng.xml</suiteXmlFile>
        </suiteXmlFiles>
      </configuration>
    </plugin>
  </plugins>
</build>
</project>

```

Step 2: Thread-Safe Driver Engine (DriverManager.java)

Manages WebDriver instances using ThreadLocal to ensure complete thread isolation during parallel execution.

Java

```
package com.framework.driver;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.firefox.FirefoxDriver;
import java.time.Duration;

public class DriverManager {
    private static final ThreadLocal<WebDriver> dr = new ThreadLocal<>();

    public static WebDriver getDriver() {
        return dr.get();
    }

    public static void setDriver(WebDriver driver) {
        dr.set(driver);
    }

    public static void unload() {
        dr.remove();
    }

    public static void initDriver(String browser) {
        WebDriver driver;
        if (browser.equalsIgnoreCase("firefox")) {
            driver = new FirefoxDriver();
        } else {
            driver = new ChromeDriver(); // Default fallback
        }
        driver.manage().window().maximize();
        driver.manage().timeouts().pageLoadTimeout(Duration.ofSeconds(30));
        setDriver(driver);
    }

    public static void quitDriver() {
        if (getDriver() != null) {
            getDriver().quit();
            unload();
        }
    }
}
```

```
}
```

Step 3: Excel Utilities Data Engine (ExcelUtils.java)

Reads test data arrays dynamically from an external Excel sheet using Apache POI.

Java

```
package com.framework.utils;
```

```
import org.apache.poi.ss.usermodel.DataFormatter;
import org.apache.poi.xssf.usermodel.XSSFCell;
import org.apache.poi.xssf.usermodel.XSSFRow;
import org.apache.poi.xssf.usermodel.XSSFSheet;
import org.apache.poi.xssf.usermodel.XSSFWorkbook;
```

```
import java.io.FileInputStream;
```

```
import java.io.IOException;
```

```
public class ExcelUtils {
```

```
    public static Object[][] getTestData(String filePath, String sheetName) {
```

```
        XSSFWorkbook workbook = null;
```

```
        Object[][] data = null;
```

```
        try (FileInputStream fis = new FileInputStream(filePath)) {
```

```
            workbook = new XSSFWorkbook(fis);
```

```
            XSSFSheet sheet = workbook.getSheet(sheetName);
```

```
            int rowCount = sheet.getPhysicalNumberOfRows();
```

```
            XSSFRow row = sheet.getRow(0);
```

```
            int colCount = row.getLastCellNum();
```

```
            data = new Object[rowCount - 1][colCount];
```

```
            DataFormatter formatter = new DataFormatter();
```

```
            for (int i = 0; i < rowCount - 1; i++) {
```

```
                XSSFRow currentRow = sheet.getRow(i + 1);
```

```
                for (int j = 0; j < colCount; j++) {
```

```
                    if (currentRow == null) {
```

```
                        data[i][j] = "";
```

```
                    } else {
```

```
                        XSSFCell cell = currentRow.getCell(j);
```

```
                        data[i][j] = formatter.formatCellValue(cell);
```

```
                    }
```

```
                }
```

```
            }
```

```
        } catch (IOException e) {
```

```

        throw new RuntimeException("Could not read Excel file: " + filePath, e);
    }
    return data;
}
}

```

Step 4: Base Page Guard Class (BasePage.java)

Encapsulates explicit waits so child page objects don't interact with raw, un-synchronized web elements.

Java

```
package com.framework.pages;
```

```

import com.framework.driver.DriverManager;
import org.openqa.selenium.By;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;

```

```
import java.time.Duration;
```

```

public class BasePage {
    protected WebDriverWait wait;

    protected BasePage() {
        this.wait = new WebDriverWait(DriverManager.getDriver(), Duration.ofSeconds(10));
    }

    protected void click(By by) {
        wait.until(ExpectedConditions.elementToBeClickable(by)).click();
    }

    protected void writeText(By by, String text) {
        WebElement element = wait.until(ExpectedConditions.visibilityOfElementLocated(by));
        element.clear();
        element.sendKeys(text);
    }

    protected String getText(By by) {
        return wait.until(ExpectedConditions.visibilityOfElementLocated(by)).getText();
    }
}

```

Step 5: Standard Page Objects (LoginPage.java & DashboardPage.java)

Java

```
package com.framework.pages;
```

```

import org.openqa.selenium.By;

public class LoginPage extends BasePage {
    private final By usernameField = By.name("username");
    private final By passwordField = By.name("password");
    private final By loginButton = By.xpath("//button[@type='submit']");

    public void login(String username, String password) {
        writeText(usernameField, username);
        writeText(passwordField, password);
        click(loginButton);
    }
}

```

Java

```
package com.framework.pages;
```

```

import org.openqa.selenium.By;

public class DashboardPage extends BasePage {
    private final By dashboardHeader = By.tagName("h6");

    public String getDashboardHeading() {
        return getText(dashboardHeader);
    }
}

```

Step 6: Extent Reporting Utility (ExtentReportManager.java)

Manages thread-safe report logging and captures failure screenshots directly from the driver pool.

Java

```
package com.framework.utils;
```

```

import com.aventstack.extentreports.ExtentReports;
import com.aventstack.extentreports.ExtentTest;
import com.aventstack.extentreports.reporter.ExtentSparkReporter;
import com.framework.driver.DriverManager;
import org.openqa.selenium.OutputType;
import org.openqa.selenium.TakesScreenshot;

public class ExtentReportManager {
    private static ExtentReports extent;
    private static final ThreadLocal<ExtentTest> testThread = new ThreadLocal<>();
}

```

```

public static void initReport() {
    if (extent == null) {
        extent = new ExtentReports();
        ExtentSparkReporter spark = new ExtentSparkReporter("reports/index.html");
        spark.config().setReportName("Automation Execution Report");
        extent.attachReporter(spark);
    }
}

public static void flushReport() {
    if (extent != null) {
        extent.flush();
    }
}

public static void createTest(String testName) {
    ExtentTest test = extent.createTest(testName);
    testThread.set(test);
}

public static ExtentTest getTest() {
    return testThread.get();
}

public static void logPass(String message) {
    getTest().pass(message);
}

public static void logFail(String message) {
    getTest().fail(message);
    String base64Screenshot = ((TakesScreenshot)
DriverManager.getDriver()).getScreenshotAs(OutputType.BASE64);
    getTest().addScreenCaptureFromBase64String(base64Screenshot, "Failure Screenshot");
}

public static void unload() {
    testThread.remove();
}
}

```

Step 7: Base & Test Execution Layer (BaseTest.java & LoginTest.java)

Java

```

package com.framework.tests;

import com.framework.driver.DriverManager;

```



```

import com.framework.utils.ExtentReportManager;
import org.testng.ITestResult;
import org.testng.annotations.*;

import java.lang.reflect.Method;

public class BaseTest {

    @BeforeSuite
    public void setupSuite() {
        ExtentReportManager.initReport();
    }

    @BeforeMethod
    @Parameters({"browser", "url"})
    public void setUp(@Optional("chrome") String browser,
@Optional("https://opensource-demo.orangehrmlive.com/") String url, Method method) {
        ExtentReportManager.createTest(method.getName() + " [" + browser + "]");
        DriverManager.initDriver(browser);
        DriverManager.getDriver().get(url);
    }

    @AfterMethod
    public void tearDown(ITestResult result) {
        if (result.getStatus() == ITestResult.FAILURE) {
            ExtentReportManager.logFail(result.getThrowable().getMessage());
        } else if (result.getStatus() == ITestResult.SUCCESS) {
            ExtentReportManager.logPass("Test passed successfully.");
        }
        DriverManager.quitDriver();
        ExtentReportManager.unload();
    }

    @AfterSuite
    public void tearDownSuite() {
        ExtentReportManager.flushReport();
    }
}

Java
package com.framework.tests;

import com.framework.pages.DashboardPage;
import com.framework.pages.LoginPage;
import com.framework.utils.ExcelUtils;

```

```

import org.testng.Assert;
import org.testng.annotations.DataProvider;
import org.testng.annotations.Test;

public class LoginTest extends BaseTest {

    @DataProvider(name = "loginData", parallel = true)
    public Object[][] getLoginData() {
        return ExcelUtils.getTestData("src/test/resources/testdata/TestData.xlsx", "LoginSheet");
    }

    @Test(dataProvider = "loginData")
    public void verifyLogin(String username, String password, String expectedHeader) {
        LoginPage loginPage = new LoginPage();
        DashboardPage dashboardPage = new DashboardPage();

        loginPage.login(username, password);
        String actualHeader = dashboardPage.getDashboardHeading();

        Assert.assertEquals(actualHeader, expectedHeader, "Dashboard text mismatch!");
    }
}

```

Step 8: Multi-Threaded Controller Configuration (testng.xml)

This runs your tests inside threads across parallelized instances based on parameters.

XML

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE java>
<suite name="Data Driven Suite" parallel="tests" thread-count="2">

    <test name="Chrome Login Tests">
        <parameter name="browser" value="chrome"/>
        <parameter name="url" value="https://opensource-demo.orangehrmlive.com/">
        <classes>
            <class name="com.framework.tests.LoginTest"/>
        </classes>
    </test>

    <test name="Firefox Login Tests">
        <parameter name="browser" value="firefox"/>
        <parameter name="url" value="https://opensource-demo.orangehrmlive.com/">
        <classes>
            <class name="com.framework.tests.LoginTest"/>
        </classes>
    </test>
</suite>

```

</test>
</suite>

5. Justifications & Strategic Alternatives

Why ThreadLocal over sequential execution?

Reasoning: In an enterprise environment, tests grow into the hundreds. Running tests sequentially creates an unacceptable CI/CD bottleneck. ThreadLocal unlocks safe concurrency, dropping total test execution times exponentially.

Alternative: You could build standard sequential runners, but this results in long build lines, scaling issues, and architectural refactoring costs later.

Why standard POM over Component POM or Fluent?

Reasoning: Standard POM establishes an incredibly low barrier to entry for incoming QA engineers. Every app viewport maps exactly to one class file.

Alternative: Component-Based POM is superior if your application contains highly intricate, repeating shared layout pieces (e.g., custom nested grid tables). However, it adds a layer of abstraction that increases initial setup complexity.

Why Base64 Embedded Images in Extent Reports?

Reasoning: Standard configurations save failure screenshots as standalone files in local paths. When sharing HTML reports through build pipes (like Jenkins or shared emails), standard images break because their paths point locally. Base64 encodes images directly inside the single HTML output file, ensuring it remains fully portable.